

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería



Proyecto Final

Docente:

- Ing. Jared López Leaña

estudiante:

- Luis Gerardo Portillo Gonzales

Asignatura / Materia: Tecnología de Base de Datos I

Turno: Medio Dia

Santa Cruz-Bolivia

2026

Introducción

Este documento cierra el proyecto de migración de la base employees, que pasó de MariaDB a PostgreSQL 18. En la actividad anterior ya había movido las tablas y los datos; en esta etapa lo que me tocaba era terminar el trabajo con las vistas y, sobre todo, demostrar con pruebas concretas que la información llegó completa y sin alteraciones.

Para eso me propuse varias cosas: comprobar que cada tabla tuviera la misma cantidad de filas en ambos motores, llevar las dos vistas de MariaDB a PostgreSQL y probarlas, buscar registros huérfanos que rompan las relaciones, comparar huellas digitales (checksums) de algunas tablas, revisar las restricciones que quedaron en el esquema y, al final, generar un backup completo de la base ya migrada. Todas las evidencias que presento son salidas en texto de los comandos y consultas que ejecuté.

Entorno de trabajo

Trabajé con dos servidores en la misma máquina: MariaDB como origen y PostgreSQL como destino, este último corriendo dentro de un contenedor Docker. Los datos de cada uno son los siguientes:

Datos de conexión del origen y del destino

Parámetro	Origen (MariaDB)	Destino (PostgreSQL)
Versión	MariaDB 11.8.9	PostgreSQL 18.6
Servidor	127.0.0.1	127.0.0.1
Puerto	3306	5432
Usuario	luis	luis
Base de datos	employees	pdb_employees
Esquema	—	employees

Para conectarme a cada uno usé estos comandos:

```
mariadb -h 127.0.0.1 -P 3306 -u luis -p employees  
psql -h 127.0.0.1 -p 5432 -U luis -d pdb_employees
```

La base de origen tiene seis tablas (departments, dept_emp, dept_manager, employees, salaries y titles) y dos vistas (current_dept_emp y dept_emp_latest_date).

Migración de tablas y datos

Las tablas las había migrado antes con pgloader, usando el archivo de instrucciones migracion.load. Lo resumo acá porque es la base sobre la que se hicieron todas las verificaciones:

```
LOAD DATABASE
FROM mysql://root:123456@localhost:3306/employees
INTO postgresql://luis:123456@localhost:5432/pdb_employees
WITH include drop, create tables, create indexes, reset sequences,
workers = 8, concurrency = 2, batch rows = 10000
SET maintenance_work_mem to '256MB', work_mem to '32MB'
CAST type datetime to timestamp drop default drop not null
using zero-dates-to-null,
type enum to text;
```

Al ejecutar pgloader migracion.load, la herramienta pasó 3.919.015 filas (unos 134,9 MB) sin un solo error, y además armó 9 índices, 6 claves primarias y 6 claves foráneas.

Comparación de conteos

Para confirmar que no faltara nada, corrí la misma consulta de conteo en los dos motores. Es una sola sentencia que une con UNION ALL un COUNT(*) por cada tabla, así en una sola salida veo las seis cantidades. En MariaDB la ejecuté así:

```
mariadb -h 127.0.0.1 -P 3306 -u luis -p employees -e "
SELECT 'departments' AS tabla, COUNT(*) AS filas FROM departments
UNION ALL SELECT 'dept_emp', COUNT(*) FROM dept_emp
UNION ALL SELECT 'dept_manager', COUNT(*) FROM dept_manager
UNION ALL SELECT 'employees', COUNT(*) FROM employees
UNION ALL SELECT 'salaries', COUNT(*) FROM salaries
UNION ALL SELECT 'titles', COUNT(*) FROM titles;"
```

En PostgreSQL usé la misma consulta con psql -c, con la única diferencia de anteponer el esquema a cada tabla (por ejemplo employees.salaries), porque pgloader las dejó en el esquema employees y no en public. Las salidas quedaron guardadas en final_conteos_mariadb.txt y final_conteos_postgresql.txt.

Cantidad de filas por tabla en origen y destino

Tabla	MariaDB	PostgreSQL	Resultado
departments	9	9	Coincide
dept_emp	331.603	331.603	Coincide
dept_manager	24	24	Coincide
employees	300.024	300.024	Coincide
salaries	2.844.047	2.844.047	Coincide
titles	443.308	443.308	Coincide

Las seis tablas tienen exactamente la misma cantidad de filas en ambos lados.

Migración de vistas

pgloader se encarga de las tablas, pero no trae las vistas, así que esta parte la hice a mano. Primero confirmé que en PostgreSQL todavía no existiera ninguna:

```
psql -h 127.0.0.1 -p 5432 -U luis -d pdb_employees -c "\dv employees.*"  
-- respuesta: No se encontraron vistas con el nombre «employees.*».
```

Qué hace cada vista

Saqué la definición original de cada una desde MariaDB con `SHOW CREATE VIEW` y guardé las salidas en `vista_latest_mariadb.txt` y `vista_current_mariadb.txt`. Leyéndolas, la lógica es la siguiente:

- `dept_emp_latest_date` agrupa la tabla `dept_emp` por empleado y se queda con la fecha de inicio y la fecha de fin más recientes de cada uno. En pocas palabras, dice cuándo empezó y terminó la última asignación de cada persona.
- `current_dept_emp` cruza `dept_emp` con la vista anterior para saber en qué departamento está (o estuvo por última vez) cada empleado.

Como `current_dept_emp` usa a `dept_emp_latest_date`, el orden de creación importa: primero tiene que existir la segunda.

Adaptación a PostgreSQL

La lógica se mantuvo igual; lo que cambié fue la sintaxis. Quité las comillas invertidas y las cláusulas propias de MariaDB (como ALGORITHM o DEFINER), agregué el esquema employees delante de cada objeto y usé alias cortos para que la unión se lea mejor. Todo quedó en el archivo `vistas_postgresql.sql`:

```
CREATE OR REPLACE VIEW employees.dept_emp_latest_date AS
SELECT emp_no,
       MAX(from_date) AS from_date,
       MAX(to_date)   AS to_date
FROM employees.dept_emp
GROUP BY emp_no;

CREATE OR REPLACE VIEW employees.current_dept_emp AS
SELECT l.emp_no, d.dept_no, l.from_date, l.to_date
FROM employees.dept_emp AS d
JOIN employees.dept_emp_latest_date AS l
  ON d.emp_no = l.emp_no
 AND d.from_date = l.from_date
 AND d.to_date = l.to_date;
```

Creación y comprobación

Cargué el archivo con `psql -f vistas_postgresql.sql` y PostgreSQL respondió dos veces `CREATE VIEW`, una por cada vista. Al volver a ejecutar `\dv employees.*` ya aparecían las dos, ambas en el esquema `employees` y con `luis` como dueño.

Pruebas de funcionamiento

Para ver que devolvieran datos con sentido, consulté las primeras diez filas de cada vista con `SELECT * ... LIMIT 10`. En la Tabla 3 se ve una parte de lo que devolvió cada una:

Muestra de filas devueltas por las vistas en PostgreSQL

Vista	emp_no	dept_no	from_date	to_date
dept_emp_latest_date	10001	—	1986-06-26	9999-01-01
dept_emp_latest_date	10008	—	1998-03-11	2000-07-31
dept_emp_latest_date	10010	—	2000-06-26	9999-01-01
current_dept_emp	10004	d004	1986-12-01	9999-01-01
current_dept_emp	10025	d005	1987-08-17	1997-10-15
current_dept_emp	10038	d009	1989-09-20	9999-01-01

Nota. La fecha 9999-01-01 es la convención de esta base para indicar que la asignación sigue vigente. Las salidas completas están en `prueba_vista_latest.txt` y `prueba_vista_current.txt`.

Después conté las filas de cada vista en los dos motores. Ambas devuelven 300.024 filas, que es justamente la cantidad de empleados: una fila por persona, tal como se espera.

Cantidad de filas de las vistas en origen y destino

Vista	MariaDB	PostgreSQL	Resultado
dept_emp_latest_date	300.024	300.024	Coincide
current_dept_emp	300.024	300.024	Coincide

Consultas de verificación

Que los conteos coincidan es una buena señal, pero no alcanza: podrían estar las mismas cantidades con datos distintos o con relaciones rotas. Por eso hice tres pruebas más.

Búsqueda de registros huérfanos

Un registro huérfano es una fila que apunta a algo que no existe; por ejemplo, un sueldo asignado a un número de empleado que no está en la tabla `employees`. Armé un script para cada motor (`verificar_huerfanos_mariadb.sql` y `verificar_huerfanos_postgresql.sql`) que revisa las seis relaciones de la base y cuenta cuántas filas quedan sin su pareja.

Registros huérfanos por relación

Relación revisada	MariaDB	PostgreSQL
dept_emp → employees	0	0
dept_emp → departments	0	0
dept_manager → employees	0	0
dept_manager → departments	0	0
salaries → employees	0	0
titles → employees	0	0

Nota. Salidas guardadas en `huerfanos_mariadb.txt` y `huerfanos_postgresql.txt`.

No apareció ningún huérfano en ninguna de las seis relaciones, ni en el origen ni en el destino. En PostgreSQL esta prueba me dio un problema de memoria la primera vez; lo explico en la sección siguiente.

Comparación por checksum

Un checksum es como una huella digital del contenido de una tabla: se calcula a partir de todos sus valores, y si cambia aunque sea un carácter, la huella sale distinta. Lo apliqué a departments y dept_manager en ambos motores y los resultados fueron idénticos, lo que confirma que no solo llegaron las mismas filas, sino con los mismos valores.

Checksums calculados en origen y destino

Tabla	Checksum (igual en MariaDB y PostgreSQL)	Resultado
departments	5e4c20a53a2dc1d1a236093c4d2cd305	Coincide
dept_manager	69c48446edd6643439446241674bd217	Coincide

Nota. Salidas guardadas en checksums_mariadb.txt y checksums_postgresql.txt.

Restricciones del esquema migrado

Por último revisé qué restricciones quedaron en PostgreSQL consultando el catálogo information_schema.table_constraints para el esquema employees:

```
SELECT table_name, constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'employees'
ORDER BY table_name, constraint_type, constraint_name;
```

La consulta devolvió 35 filas. Entre ellas están las 6 claves primarias y las 6 claves foráneas originales (dept_emp_ibfk_1, dept_emp_ibfk_2, dept_manager_ibfk_1, dept_manager_ibfk_2, salaries_ibfk_1 y titles_ibfk_1). El resto son restricciones de tipo CHECK: así es como PostgreSQL muestra en este catálogo las columnas marcadas como NOT NULL, no son reglas nuevas. La salida completa está en restricciones_postgresql_final.txt.

Problema encontrado y cómo lo resolví

La primera vez que corrí la búsqueda de huérfanos en PostgreSQL, la consulta se cortó con este error, y se repitió en un segundo intento:

```
ERROR: could not resize shared memory segment  
"/PostgreSQL.2211827842" to 8388608 bytes: No space left on device
```

El mensaje dice que PostgreSQL intentó reservar 8 MB de memoria compartida y no encontró espacio. Me llamó la atención porque en la máquina había de sobra, así que fui revisando por partes:

Revisión de la memoria compartida

Dónde revisé	Comando	Espacio total	Disponible
Máquina anfitriona	<code>df -h /dev/shm</code>	1,9 GB	1,9 GB
Contenedor de PostgreSQL	<code>docker exec postgresql df -h /dev/shm</code>	64 MB	63 MB

Ahí estaba la causa. Docker le asigna por defecto solo 64 MB de memoria compartida a cada contenedor, sin importar cuánta tenga la máquina. Las consultas sobre tablas grandes, como `salaries` con casi tres millones de filas, hacen que PostgreSQL reparta el trabajo entre varios procesos en paralelo, y esos procesos se comunican usando justamente esa memoria compartida. Con tan poco espacio, no alcanzaba.

La solución fue pedirle a PostgreSQL, solo para esa sesión, que no trabajara en paralelo y que usara menos memoria por operación. Volví a correr el script así:

```
psql -h 127.0.0.1 -p 5432 -U luis -d pdb_employees -v ON_ERROR_STOP=1 \  
-c "SET max_parallel_workers_per_gather = 0; SET work_mem = '4MB';" \  
-f verificar_huerfanos_postgresql.sql
```

Con ese ajuste la consulta terminó sin problemas y devolvió cero huérfanos en las seis relaciones. Lo bueno es que el cambio afecta solo a esa sesión y no toca ni los datos ni la configuración del servidor. Si esto se repitiera seguido, la solución de fondo sería levantar el contenedor con más memoria compartida, usando la opción `--shm-size` de Docker.

Backup final de PostgreSQL

Con todo verificado, generé una copia de seguridad completa de la base migrada.

Usé el formato personalizado de `pg_dump` (opción `-Fc`), que sale comprimido y permite restaurar objetos de forma selectiva:

```
pg_dump -h 127.0.0.1 -p 5432 -U luis -Fc -f pdb_employees_final.dump pdb_employees
```

El archivo `pdb_employees_final.dump` pesa unos 35 MB, bastante menos que los 134,9 MB de datos originales gracias a la compresión. Para comprobar qué tiene adentro sin restaurarlo, listé su contenido con `pg_restore -l`:

Datos principales del backup generado

Característica	Valor
Fecha de creación	26 de septiembre de 2026, 16:39
Base de datos	pdb_employees
Formato	Personalizado (CUSTOM), comprimido con gzip
Versión del servidor y de <code>pg_dump</code>	18.6
Entradas en el índice del backup	35
Tamaño	Aproximadamente 35 MB

En el listado aparecen el esquema `employees`, las seis tablas con su estructura y los datos de las seis tablas. Además, filtrando con `pg_restore -l pdb_employees_final.dump | grep -i VIEW`, confirmé que también están las dos vistas que creé a mano: `dept_emp_latest_date` y `current_dept_emp`. Es decir, el backup guarda la base completa tal como quedó después de la migración.

Conclusiones

La migración de la base `employees` de MariaDB a PostgreSQL 18 quedó comprobada desde varios ángulos, y todos dieron el mismo resultado. Las seis tablas tienen exactamente

la misma cantidad de filas en ambos motores, y los checksums de departments y dept_manager son idénticos, lo que muestra que el contenido llegó sin cambios.

Las dos vistas, que pgloader no migra, las pasé a mano: tomé su definición en MariaDB, ajusté la sintaxis a PostgreSQL, las creé y las probé. Ambas devuelven 300.024 filas en origen y destino. La búsqueda de huérfanos dio cero en las seis relaciones y las claves primarias y foráneas siguen en su lugar, así que la integridad de la base se mantuvo.

Lo que más me dejó esta etapa fue el error de memoria compartida. Me enseñó que el entorno donde corre el motor también cuenta: el servidor tenía memoria de sobra, pero el contenedor estaba limitado a 64 MB. Entender eso me permitió resolverlo con un ajuste de sesión, sin tocar los datos. Cerré el trabajo con pdb_employees_final.dump, un backup comprimido de unos 35 MB que contiene las tablas, los datos y las dos vistas.

Archivos del proyecto

La carpeta de entrega contiene los siguientes archivos, agrupados según la parte del trabajo a la que pertenecen:

Archivos que forman parte del entregable

Grupo	Archivos
Informe y respaldo	EntregableFinal_Luis_Gerardo_Portillo_Gonzales.docx, README.md, pdb_employees_final.dump
Conteo de tablas	final_conteos_mariadb.txt, final_conteos_postgresql.txt
Definición de vistas	vista_latest_mariadb.txt, vista_current_mariadb.txt, vistas_postgresql.sql
Creación y prueba de vistas	evidencia_creacion_vistas.txt, verificacion_vistas_postgresql.txt, prueba_vista_latest.txt, prueba_vista_current.txt
Conteo de vistas	conteos_vistas_mariadb.txt, conteos_vistas_postgresql.txt
Huérfanos	verificar_huerfanos_mariadb.sql, verificar_huerfanos_postgresql.sql, huerfanos_mariadb.txt, huerfanos_postgresql.txt
Checksums	checksums_mariadb.txt, checksums_postgresql.txt
Restricciones	restricciones_postgresql_final.txt

